

Masks Are Length Promises: How Diffusion-LM Agents Coordinate Parallel Tool Calls

Anonymous Author(s)

Submission Id: «OpenReview submission id»

Abstract

A tool-calling agent often issues several calls in one turn. Diffusion language models (dLLMs) commit many tokens per denoising step, which makes them attractive decision cores for such agents, and raises a coordination worry: tokens committed in the same step cannot see each other, so parallel calls may collide or bind each other’s arguments. We treat a dLLM agent’s parallel turn as simultaneous decisions by the value slots of a fixed call skeleton, and test what these decisions coordinate on, using the parallel calls of the Berkeley Function Calling Leaderboard. When the request orders its entities, the slots coordinate on mention order: committing 16 tokens per step instead of one costs Dream-7B 5 points of set accuracy, also where slot lengths cannot tell the calls apart. When nothing tells the calls apart, slots committed in the same step collide, in 100% of open choose-N requests against 0% one token at a time, **TBD[like teams of LLM agents that act at once without seeing each other; turn-taking prevents it]**. The costliest failures come from the action interface instead: the models read the number of masks in a slot as the length of its value. One extra mask per slot drops set accuracy from 90% to 16%, and swapping slot lengths between sibling calls makes 79% of the affected slots take the sibling value that fits, even one token per step. With estimated slot lengths, decoding one token per step makes about four times as many cross-call errors as decoding 16 tokens per step with exact lengths: errors an evaluator would blame on parallel decoding. A dLLM agent needs its slot lengths under control before parallelism becomes its problem.

Keywords

LLM agents, tool use, parallel action selection, coordination without communication, focal points, diffusion language models

ACM Reference Format:

Anonymous Author(s). 2027. Masks Are Length Promises: How Diffusion-LM Agents Coordinate Parallel Tool Calls. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 4 pages.

1 Introduction

[Planned: (1) Agents issue several tool calls per turn, a joint action; dLLMs commit many tokens per step and look like natural decision cores; the worry in coordination terms: same-step tokens are simultaneous moves without communication, so calls may collide or cross-bind. Lu et al. (ACL 2026) report that dLLM agents fail at tool calling and

blame diffusion noise, without a controlled test. (2) The uncontrolled confound: a skeleton interface must fix slot lengths before decoding, and value lengths are unknown, so mismatch is the norm (fixed budgets → surplus; per-argument maximum → a sibling fits; estimates → both). (3) Our approach: a parallel turn as simultaneous decisions; predictions from coordination cues; independent control of the protocol (k , blocks) and the interface (slot length); a team-of-agents control. (4) Findings and contributions (structure.md §2). Figure 1: parallel_20 under exact / +1 / swap lengths, one token per step.]

2 Related Work

[Planned: LLM agents and parallel tool calls (LLMCompiler; BFCL parallel categories); dLLM agents (Lu et al. 2026; DLLM Agent; DLLM-Searcher; Agents of Diffusion, AAMAS 2026). Coordination without communication (Schelling focal points; Lewis conventions; dispersion games; zero-shot coordination, Other-Play): we apply these notions to the parallel decisions inside one agent and compare with a team of real agents. Parallel decoding and length in dLLMs, compressed: factorization error (Fast-dLLM, ParallelBench, EB-Sampler); variable length (CAL, single-span only; ρ -EOS; DAEDAL; S^3); constrained decoding (Dang & Ermon; Mündler et al.).]

3 A dLLM Agent’s Parallel Turn as Simultaneous Decisions

3.1 The Agent and Its Action Interface

A tool-calling agent receives a request q and a set of tool schemas $\mathcal{F}$, and acts by issuing a set of calls $A = \{a_1, \dots, a_n\}$, where each call $a_i = (f_i, \theta_i)$ names a tool $f_i \in \mathcal{F}$ and binds its arguments θ_i . When $n > 1$, the turn is a joint action. We score it as a set, as BFCL does [2]: the turn is correct if its calls can be matched one to one with the reference calls, whatever their order.

The agent’s decision core is a masked diffusion language model (dLLM). It writes A as a JSON array onto a canvas of mask tokens and fills the canvas over several denoising steps. To make every action well formed, the agent’s interface fixes the syntax of the turn in advance with a *skeleton*: the number of calls, the tool names, the argument names, and all brackets and quotes are written onto the canvas, and only the argument values are masked. Each value occupies a *slot* of ℓ mask tokens. The slots that hold the same argument in different calls of the same tool form a *sibling group*; these are the slots that can be confused with each other. Our skeleton is an oracle, built from the reference calls, so that every remaining error is an error in a value. Section 9 discusses what the oracle hides.

A slot need not be filled. A value ends when the model writes its closing token (the closing quote for a string; for other types, a comma or closing brace outside any bracket of the value), and the rest of the slot is filled with padding. Padding is forced and is not counted as a decision. The autoregressive (AR) reference agent uses the same skeleton and the same rule: it writes each value until its



This work is licensed under a Creative Commons Attribution International 4.0 License.

closing token and is cut off at the slot length, but it is never made to fill a slot. Surplus masks therefore cannot change its output; this is a property of the design, not a finding.

3.2 Decoding as a Protocol Among Slot-Level Deciders

At each denoising step, the dLLM predicts every masked position from the current canvas and commits some of them. A position committed in a step is chosen from its own predicted distribution given the canvas *before* that step, so the positions committed in the same step cannot see each other’s choices. We call the masked positions *slot-level deciders*. The decoding schedule then acts as a protocol that says which deciders move when:

- With k tokens per step in confidence order, the k most confident masked positions move together. With $k = 1$, one decider moves at a time and sees every earlier commitment; we call this *one token per step*. With $k \geq 2$, several deciders, possibly in different calls, move simultaneously.
- Left-to-right decoding ($k = 1$) fixes the order of moves instead of letting confidence choose it. A confidence threshold τ [5] lets every position whose top probability exceeds τ move at once.
- LLaDA2.0 decodes in blocks of 32 tokens, with bidirectional attention inside a block and causal attention across blocks; a block is finished before the next one starts. Slots in different blocks are never committed together, and the later slot sees the earlier one. In our skeletons only 3.1% (parallel) and 2.3% (parallel_multiple) of the pairs of sibling slots share a block, so LLaDA2.0’s calls take turns at any k .

3.3 Coordination Cues and Predictions

The sibling slots of a turn face a coordination problem. Each slot must take the value that belongs to its own call, for instance the i -th of several cities, and a slot moving in the same step as its siblings cannot see what they take. Under greedy decoding, deciders with near-identical views make identical choices. What can set them apart is a cue that every slot can read from the shared canvas and that tells each slot which value is its own. We consider three such cues:

- (1) *Mention order*. The request names the entities in some order, and each slot sees the position of its call on the canvas. Taking the i -th mentioned entity for the i -th call is a convention that all slots can follow without communicating, a focal point in the sense of Schelling [4].
- (2) *Slot position*. A slot’s position on the canvas is a role label. A label helps only if something orders the candidate values: “you are the second call” does not say which of all US cities is the second.
- (3) *Slot length*. Sibling values usually differ in length, so the number of masks in a slot can tell the slot which value it holds, provided the model reads the mask count as the length of the value.

These cues give three predictions, which Sections 4–7 test.

- P1** *Order is a focal point*. When the request orders its entities, the slots take the i -th entity for the i -th call at any k , so

committing many tokens per step costs little, even when slot lengths cannot tell the calls apart.

- P2** *Symmetric calls collide*. When nothing tells the calls apart, slots committed in the same step take the same value, while slots that take turns do not. A team of LLM agents that act at the same time without seeing each other should collide in the same way.
- P3** *Length is a promise*. A slot reads its mask count as the length of its value. A slot of the wrong length therefore receives a value of the length it promises: a longer version of its own value, or a sibling’s value that fits. This happens at any k , including one token per step.

3.4 Models, Data, and Measures

Models. Dream [6] (Dream-v0-Instruct-7B) is a dLLM with full bidirectional attention, initialized from Qwen2.5-7B. LLaDA2.0 [1] (LLaDA2.0-mini, a mixture of experts with 16B parameters, 1.4B active) is a block-diffusion dLLM trained for tool use. The AR reference agent is Qwen2.5-7B-Instruct [3], the model Dream starts from. All models see the same system prompt, which lists the tools as JSON and asks for a JSON array of calls, and all decode greedily.

Data. We use the `parallel` and `parallel_multiple` categories of BFCL v4 [2]: 400 requests with 2 to 8 calls (2.9 on average) and 3,063 value slots. LLaDA2.0’s experiments with wrong slot lengths use a subset of 100 requests, 50 evenly spaced in each category. For requests in which nothing tells the calls apart, we add 105 *choose-N* requests (Section 5).

Slot lengths. *Exact*: each slot has as many masks as its own reference value has tokens. *Surplus +s*: every slot gets $s \in \{1, 2, 4, 8\}$ extra masks. *Swap*: slot lengths are rotated by one position within each sibling group, on the requests where this changes at least one slot (165 for Dream, 34 for LLaDA2.0). *Estimate*: lengths predicted by the model itself in one forward pass, without reference values (Section 8).

Measures. *Set accuracy* is the BFCL verdict. The *cross-call error rate* (CCER) is the share of requests with at least one cross-call error, found by an optimal one-to-one matching of predicted and reference calls: a duplicated or missing call, a value that belongs to another call (cross-binding), a string spliced from two calls’ values (chimera), or an argument that the reference shares across calls but the prediction does not. At the slot level, we classify each value as the slot’s own reference value, a sibling’s value (*fit* if the sibling’s value has exactly the slot’s length), an *overflow* (the own value followed by more), a truncation of the own value, another value, or missing.

4 Ordered Requests: Same-Step Commits Coordinate on Mention Order

We first test **P1** on the BFCL requests with exact slot lengths (Table 1).

Committing many tokens per step costs little. Committing 16 tokens per step instead of one lowers Dream’s set accuracy from 89.8% to 84.5% and LLaDA2.0’s from 87.8% to 79.8%, while cutting the forward passes per request by factors of 13 and 7. The confidence

Table 1: Committing more tokens per step, with exact slot lengths (BFCL parallel and parallel_multiple, 400 requests). Set accuracy and CCER in %; CCER (sym.) on the requests whose slot lengths cannot tell the calls apart (213 for Dream, 216 for LLaDA2.0); order: share of symmetric parallel requests whose i -th call took the i -th mentioned entity; fwd: forward passes per request. LTR: left to right, one token per step; τ : commit every position above confidence 0.9. Qwen2.5 (autoregressive) in the same skeleton: 86.5% set accuracy, 1.3% CCER.

	$k=1$	2	4	8	16	LTR	τ
<i>Dream (full attention)</i>							
Set acc.	89.8	88.0	87.0	86.3	84.5	89.3	89.0
CCER	1.3	1.3	1.5	1.8	2.0	1.0	1.3
CCER (sym.)	1.9	1.9	2.3	2.8	2.8	1.4	1.9
Order	95.2	95.2	95.0	96.1	96.1	97.5	95.1
Fwd	28.0	14.2	7.3	3.9	2.2	27.8	1.9
<i>LLaDA2.0 (32-token blocks)</i>							
Set acc.	87.8	87.5	86.5	83.3	79.8	–	–
CCER	2.5	2.8	3.3	4.0	4.5	–	–
CCER (sym.)	0.9	0.9	1.4	1.4	1.9	–	–
Order	95.1	96.3	96.3	95.1	93.8	–	–
Fwd	27.8	14.8	8.3	5.1	3.9	–	–

threshold, which lets every confident position move at once, keeps Dream at 89.0% with 1.9 forward passes per request, fewer than with $k = 16$. Cross-call errors stay rare at every k : at most 2.0% of requests for Dream and 4.5% for LLaDA2.0, against 1.3% for the AR agent. At the slot level, 95.4% of Dream’s slots hold their own value with one token per step and 92.0% with 16.

The calls are not told apart by their lengths. With exact lengths, sibling slots of different lengths could be told apart by their length alone, which would keep cross-call errors low without any coordination. On the requests in which every sibling group has slots of equal length (213 for Dream, 216 for LLaDA2.0), the cross-call error rate stays as low: from 1.9% to 2.8% for Dream and from 0.9% to 1.9% for LLaDA2.0 between $k = 1$ and $k = 16$.

Mention order is the focal point. Consider the length-symmetric requests of the parallel category, in which several calls of one tool differ only in their entities. There, the i -th call takes the i -th mentioned entity in 95.0–97.5% of Dream’s and 93.8–96.3% of LLaDA2.0’s requests, with no trend in k . Each slot can work out which entity is its own from the request and its own position, so it does not need to see what its siblings commit. Because BFCL lists the reference calls in mention order and our skeleton lays the calls out in that order, this convention is also the correct one.

Where the cost of parallelism lies. The cost falls mostly inside single values, not between calls. Dream gets 62 requests wrong at $k = 16$ against 41 at $k = 1$. Output that no longer parses grows from 4 to 21 requests: tokens of one value committed together that do not fit together. Requests with cross-call errors grow only from 5 to 8. LLaDA2.0’s calls take turns at any k , because its sibling slots almost never share a block (Section 3.2). Of its 32 additional failures,

18 are single-call value errors, 8 cross-call errors, and 6 output that does not parse.

When the request orders its entities, simultaneous moves need no communication: the order is a convention that every slot shares, as P1 predicts.

5 Symmetric Requests: Same-Step Slots Collide Like Agents That Cannot Talk

[Planned: Test P2. Choose- N requests (list: n of $n+3$ listed one-token cities; open: “ n different US cities of your choice”). Single canvas: Dream duplicates in 10–15% (list) and 100% (open) of requests at $k \geq 2$, 0% at $k = 1$; pair by pair, Dream’s same-step city slots take the same city in 70% of open pairs (105/150), LLaDA2.0’s in 6% (2/32), and LLaDA2.0’s other pairs sit in different blocks (78%). Team of n LLM agents (Qwen2.5, Dream) under four protocols: simultaneous or turn-taking $\times$ anonymous or labeled (experiment B, TBD). Table 2 sets the two side by side. Open-variant validity is limited by the one-token slot; only duplicates are reported there.]

6 Masks Are Length Promises

[Planned: Test the first half of P3. Figure 2a: set accuracy against surplus s (one extra mask: 89.8% $\rightarrow$ 15.5% for Dream; slot level 95.4% $\rightarrow$ 34.0%; on the same scale, k costs 5.3 points and $s=1$ costs 74.3). What fills the extra masks: overfill (35.1% of slots at $s=1$; \$500,000 becomes \$5,000,000), sibling copies only 0.5%. Figure 2b: teacher-forced probe, $P(\text{closing token})$ right after the reference value with s masks left: 0.010 / 0.033 / 0.254 / 0.740 for Dream; near misses hurt most. Closing token versus a null token (S^3).]

7 Length Errors Masquerade as Coordination Failures

[Planned: Second half of P3, the main contribution. Swap (Table 3): 79.4% of the swapped slots take the sibling value that fits, one token per step (0% with exact lengths), in both directions; LLaDA2.0 47.5% on 34 requests. The length cue outranks the order cue about 15 to 1. Table 4: cross-call labels under one token per step with wrong lengths versus 16 tokens per step with exact lengths (estimated lengths: 7.5% vs 2.0% for Dream, 14% vs 3% for LLaDA2.0; chimeras and duplicates appear one token per step). What the wrong actions do (wrong entity, repeated action, amounts off by 10 $\times$). An evaluation protocol for dLLM agents.]

8 Designing and Evaluating dLLM Agents

[Planned: Table 5: a logit bias toward closing tokens rescues large surplus only ($s=8$: 57.0% $\rightarrow$ 72.3% at $\beta=2$; $s=2$ stays near 3%); a one-forward closing-hazard estimate of slot lengths reaches 42.2% (Dream) and 39.0% (LLaDA2.0) against about 90% with exact lengths. Recommendations for the interface (never a near miss), the protocol (symmetric calls take turns), and evaluation (control length before blaming parallelism).]

9 Discussion, Limitations, and Conclusion

[Planned: The oracle skeleton fixes the number and names of the calls (free generation: 75.2% set accuracy for Dream vs 88.0% with the skeleton at $k=2$); two dLLMs; LLaDA2.0 subsets of 100 and 34

requests; greedy decoding only; the one-token slot of the open choose- N variant; no S^3 -style prompted null baseline; skeleton tokenization differs from free generation. Beyond one agent: parallel decisions are safe when a shared convention assigns roles and collide when roles are symmetric; misleading shared signals mis-bind even sequential decisions; the same holds for multi-agent LLM systems that claim subtasks in parallel.]

Use of AI Tools

We used Claude Opus 5.5 (Anthropic) as an assistant throughout this project: to help design the experiments, to write and run the experiment and analysis code, to analyse the results, and to draft and revise the text. The prompts used to run the experiments are included in the supplementary material, and every number in the paper is produced by the released scripts from the released model outputs.

References

- [1] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, et al. 2025. LLaDA2.0: Scaling Up Diffusion Language Models to 100B. *arXiv preprint arXiv:2512.15745* (2025).
- [2] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)* (PMLR, Vol. 267).
- [3] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [4] Thomas C. Schelling. 1960. *The Strategy of Conflict*. Harvard University Press, Cambridge, MA.
- [5] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, et al. 2025. Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding. *arXiv preprint arXiv:2505.22618* (2025).
- [6] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7B: Diffusion Large Language Models. *arXiv preprint arXiv:2508.15487* (2025).